Understanding Airflow's Architecture

In this train, we'll delve into the core architectural components of Apache Airflow, exploring how they interact to manage and schedule workflows.

Learning objectives

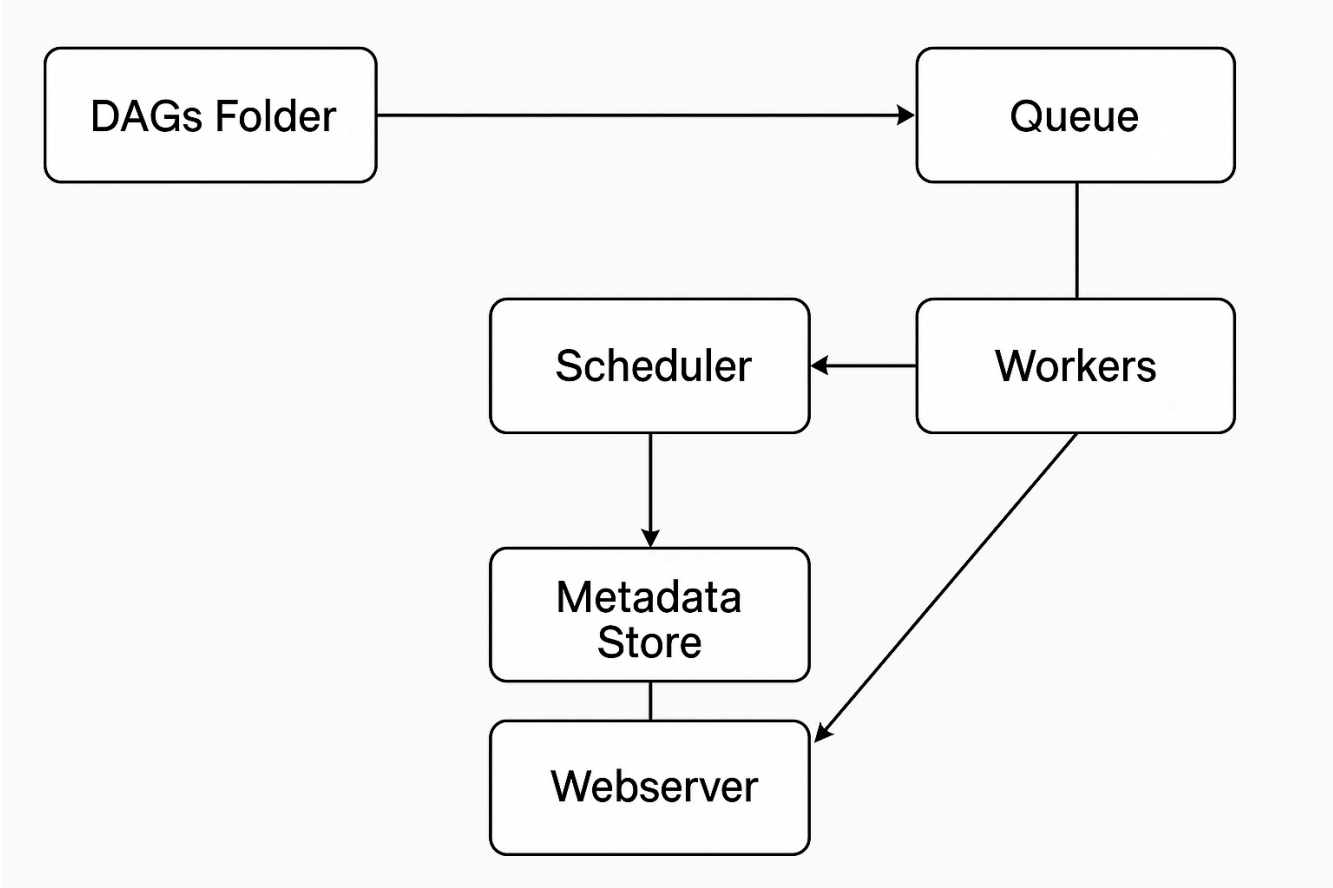
> - By the end of this train, you should understand the core architectural components of Airflow.
> - Understand the role of DAGs, the Scheduler, Workers, Webserver, and Metadata Store in Airflow.
> - Be able to describe how each component contributes to executing and monitoring data pipelines.

Core Components of Airflow

Apache Airflow comprises several critical components that together enable orchestration of complex data workflows. Below is a breakdown of each:

- **DAGs Folder:** Contains all Python script files that define the DAGs. This is the central place where users store and manage their DAG definitions, which are interpreted by the Scheduler.
- **Scheduler:** Continuously scans DAG files in the DAGs folder, evaluates scheduling intervals, and submits ready tasks to the queue after verifying all dependencies are met.
- **Workers:** Execute tasks from the queue. Depending on the executor configured (Sequential, Local, Celery, Dask, Kubernetes), the tasks are handled differently to suit scalability needs.
- **Queue:** Holds tasks that are ready for execution. The queue ensures that task dependencies are respected and provides coordination between the Scheduler and Workers.
- **Metadata Store:** A database (often Postgres or MySQL) that stores the state of DAGs, tasks, logs, and configuration. It enables observability and historical tracking of workflows. This database enables Airflow to maintain state across restarts and replays, ensuring reproducibility and auditability.
- **Webserver:** A Flask-based UI service that provides users an interface to inspect DAGs, monitor task progress, view logs, and manually trigger or rerun tasks.

How Airflow Components Work Together



1. The user defines data pipelines in Python scripts as DAGs and places them in the DAGs folder.
2. The Scheduler parses these DAGs regularly and evaluates their schedules and task dependencies.
3. Once a task is ready (i.e., all upstream dependencies are met), it is added to the execution queue.
4. The configured Executor assigns these tasks to available Workers, which run them and return results.
5. All statuses and metadata are written to the Metadata Store, making them accessible via the Webserver.
6. Through the Webserver UI, users can monitor workflow execution, diagnose failures, and maintain pipelines.

Conclusion

Understanding the architectural components of Apache Airflow provides a strong foundation for managing robust and scalable data workflows. From defining DAGs in Python to scheduling, executing, and monitoring tasks via Airflow's modular components, each element plays a vital role in maintaining pipeline integrity.

Key Takeaways

- **Takeaway 1:** Apache Airflow uses Directed Acyclic Graphs (DAGs) to define workflow structures.
- **Takeaway 2:** The Scheduler is the decision-maker that determines when tasks should run.
- **Takeaway 3:** Workers perform the tasks, while the Metadata Store logs execution details.
- **Takeaway 4:** The Webserver provides a user-friendly interface to monitor and manage workflows.
- **Takeaway 5:** Airflow supports scalable and distributed execution through various executor options.